MGIMO Hub

Как мы собрали цифровой деканат с командой ИИ-агентов

Описание процесса для хакатона «Цифровой деканат будущего»: от постановки задачи до работающего кабинета студента и деканата, развёрнутого по ссылке.

~16 ч
от ТЗ до сайта по ссылке, одна сессия

37
ИИ-агентов: 12 в разработке, 22 в QA, 3 проверяющих

29 тыс.
строк TypeScript, весь код написан ИИ

46 + 80
проверок безопасности и e2e-сценариев

Прототип: https://34-88-153-35.sslip.io

Ноябрь 2026

Коротко

Что внутри

MGIMO Hub: личный кабинет студента и рабочее место учебного офиса в одном продукте. Внутри работает ИИ-ассистент «Атташе», который отвечает по данным самого студента и готовит черновики заявок. Весь код, тексты, дизайн-система, тесты и развёртывание сделаны ИИ-агентами. Человек ставил задачи, принимал решения и проверял результат.

01 Проблема и почему эти разделы

02 Что получилось: кабинет студента и деканата

03 Полный перечень ИИ-инструментов

04 Как была выстроена работа с агентами

05 Ключевые промпты по этапам

06 Задания субагентам и оркестрации

07 Ошибки ИИ, как мы их ловили, финальная проверка

08 Удачные и неудачные варианты

09 ИИ внутри продукта: Атташе

10 Вход, данные и защита простыми словами

11 Архитектура

12 Роль автора

13 Как проверить за 5 минут

Доступ для жюри

Роль	Логин	Пароль	Что посмотреть
Студентка, МО, 3 курс	`student.anna`	`Demo-Anna-26`	справки, документы с QR, повышенная стипендия под вопросом
Студент, МЭО, 2 курс, контракт	`student.timur`	`Demo-Timur-26`	долг, направление на пересдачу, график оплаты
Студентка, ФЛМК, 1 курс	`student.leyla`	`Demo-Leyla-26`	пустые состояния, кампус Одинцово
Студенческий офис, все факультеты	`staff.office`	`Demo-Office-26`	очередь, заявки, аналитика, пересдачи
Учебный офис МЭО	`staff.meo`	`Demo-Meo-26`	видит только свой факультет

На странице входа есть «Быстрый вход для жюри»: любой аккаунт открывается одним касанием. Все люди, оценки и документы вымышлены.

01 · Проблема

Учёба разбросана по пяти местам, а справка требует очереди

Автор не учится в МГИМО, поэтому однокурсников не опрашивали. Вместо этого два ИИ-агента-исследователя параллельно собрали открытые источники: отзывы студентов, статьи о цифровизации вузов, бенчмарк чужих кабинетов, факты о МГИМО. По ним построены синтетические персоны с пометкой «сгенерировано ИИ» и анкета для реальной проверки.

Что нашли в исследовании	Какое решение приняли	Где в продукте
Студенты каждый день открывают только расписание, главная страница кабинетов почти не используется	Главный экран отвечает на «куда идти, что изменилось, что сдать»	«Сегодня»: пара сейчас с таймером, изменения, дедлайны, заявки
Справку ищут не там, где она лежит, половина не находит раздел	Заказ в одно касание, названия словами студента, данные из профиля подставляются сами	«Заявки», кнопка «Справка» на главной
Запросы в деканат теряются, статус непонятен	Статус как у посылки: этапы, срок, ответственный, история, уведомление на каждом шаге	Карточка заявки и очередь учебного офиса
Балльная система непрозрачна	Одна фраза «как у меня дела» и калькулятор «что если»	«Учёба», карточка дисциплины
Переносы и замены узнают последними	Изменения подсвечены «было / стало», push-уведомление всей группе	«Расписание», деканат: «Изменить расписание»
Первокурсники не знают, к кому идти	ИИ-ассистент как первая линия, по своим данным	«Атташе»
Расписание не открыть в метро, иностранцам сложно по-русски	Приложение на телефоне с офлайн-режимом, английский интерфейс	PWA, переключатель языка

Почему именно эти разделы

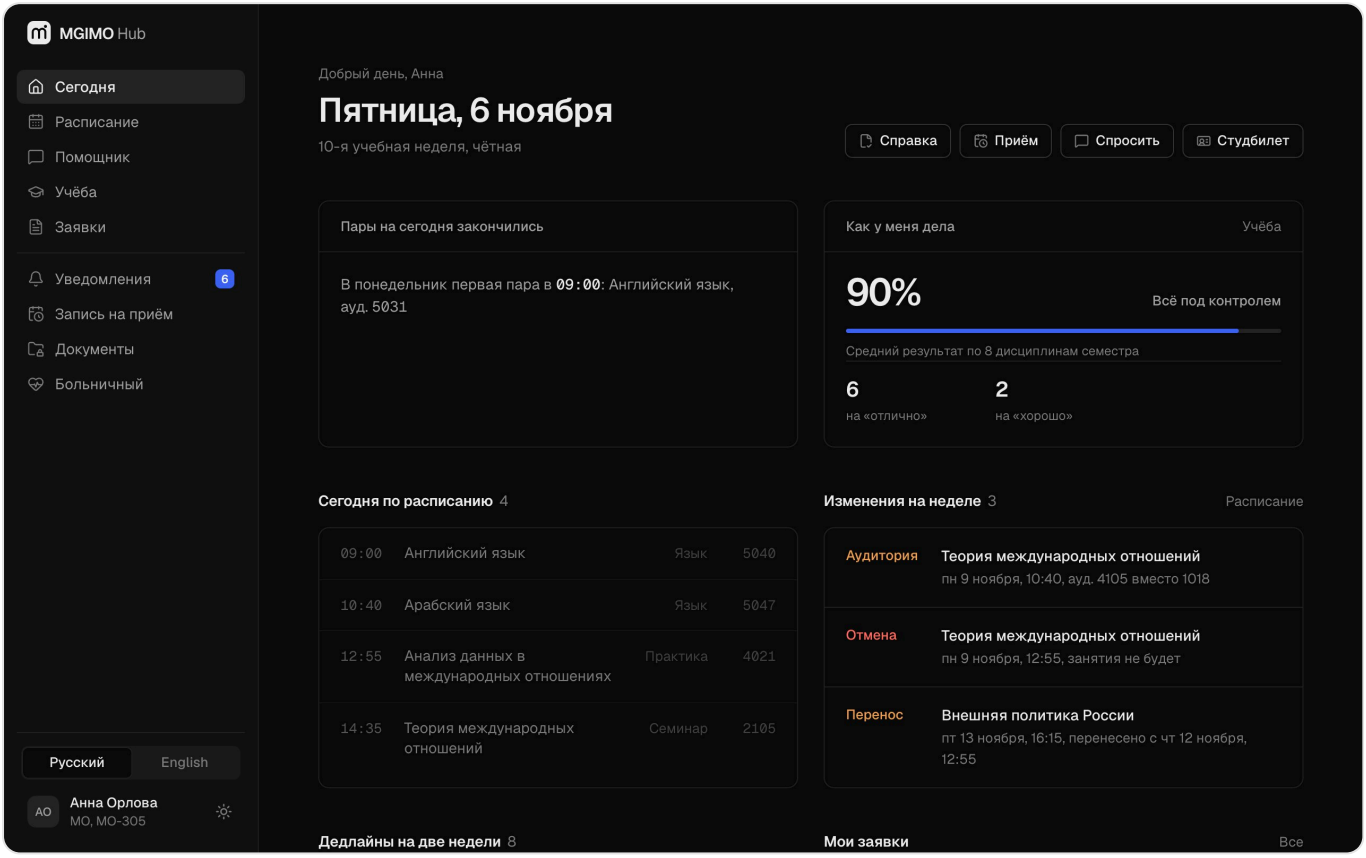
Обязательное ядро ТЗ (вход и профиль, расписание, успеваемость, заявки со статусом, уведомления) мы сделали полностью. Дополнительные разделы выбирали по одному правилу: **что сегодня заставляет студента идти в деканат ногами**. Так появились запись на приём вместо живой очереди, «Я заболел» с письмом преподавателю, все справки с QR в одном месте, оплата для контрактников и пересдачи одним действием. Для деканата: радар рисков, аналитика, журнал действий и карточка студента, чтобы цикл «студент спросил, офис ответил» замыкался в одном продукте.

> **Главная идея.** Не «сайт с разделами», а помощник, который сам говорит студенту, что сегодня важно, и превращает поход в деканат в два касания на телефоне.

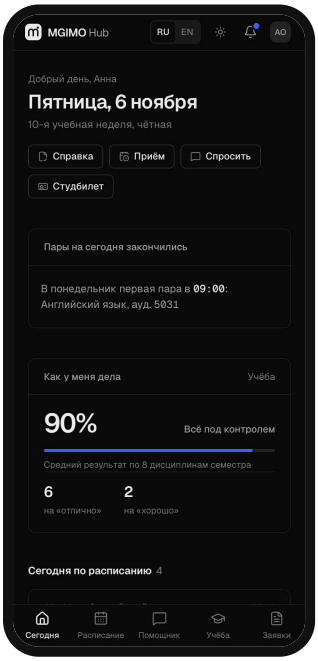
02 · Продукт

Что получилось

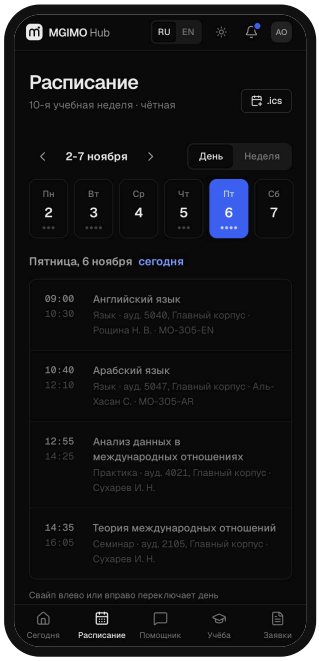
Два кабинета в одном продукте: 11 разделов для студента и 8 для учебного офиса. Всё работает на телефоне от 360 px, в светлой и тёмной теме, на русском и английском.



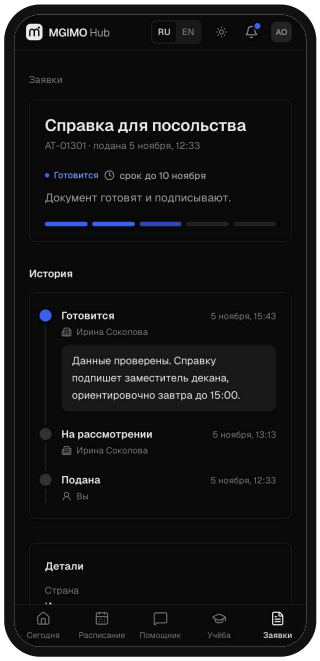
«Сегодня»: что идёт сейчас и сколько осталось, как дела с учёбой, изменения на неделе, дедлайны, статусы заявок и новости деканата.



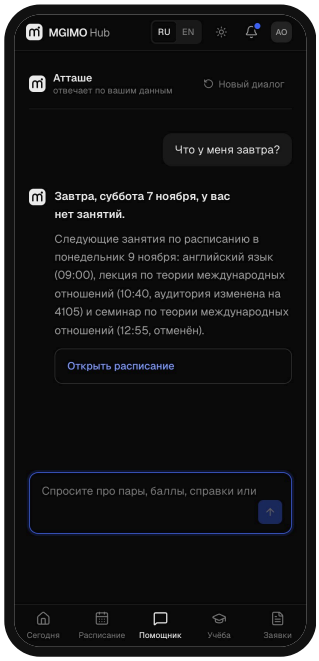
Главный экран на телефоне



Расписание с изменениями



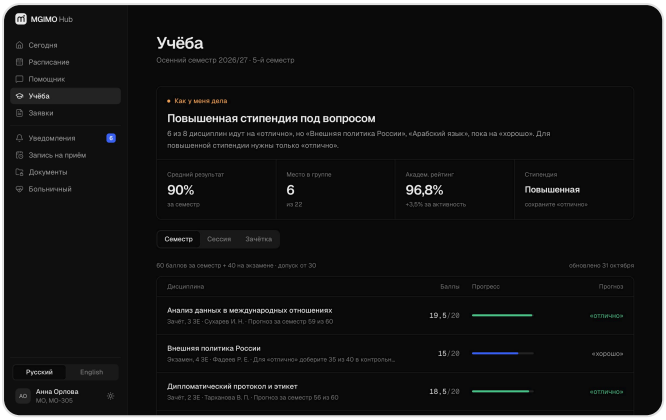
Статус заявки как у посылки



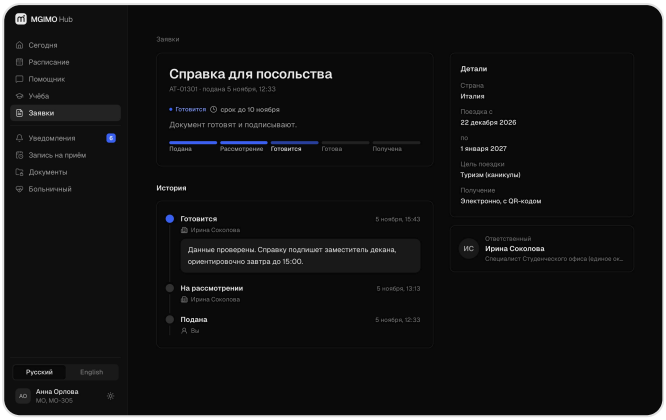
Атташе отвечает по своим данным

02 · Кабинет студента

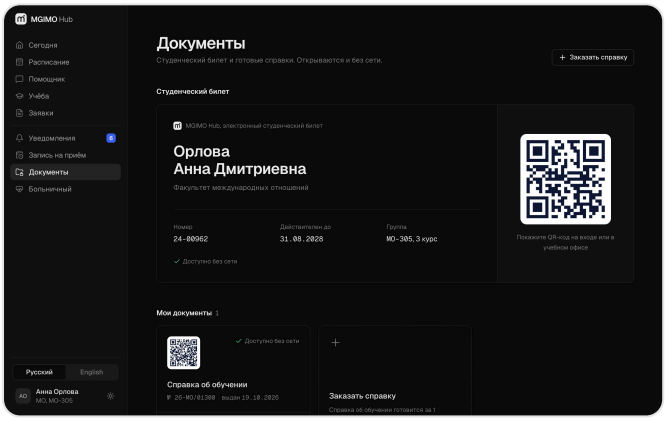
Всё про учёбу в одном месте



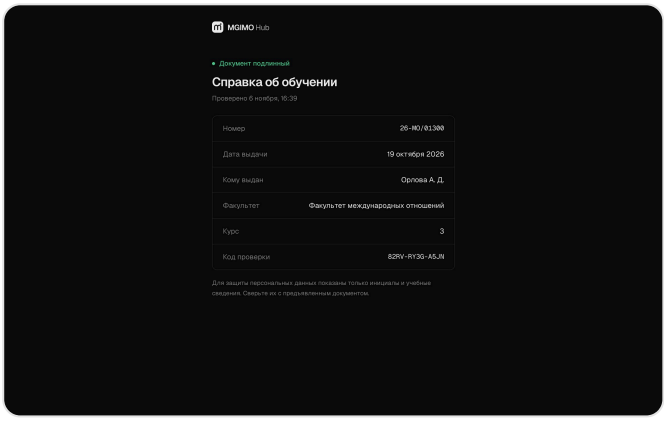
«Учёба»: одна фраза «как у меня дела», рейтинг, место в группе, баллы по каждой дисциплине и прогноз.



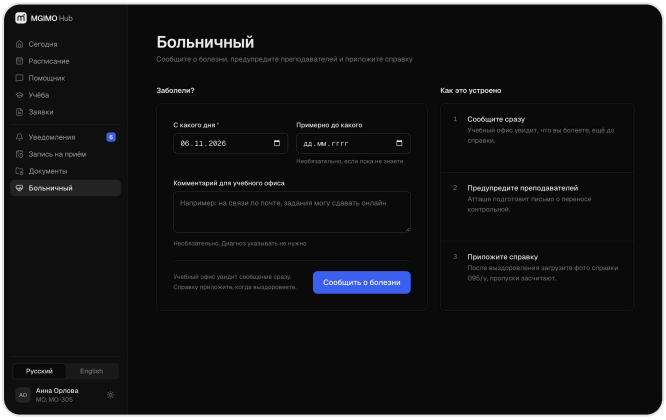
Заявка: этапы, срок, кто отвечает, история и сообщения учебного офиса.



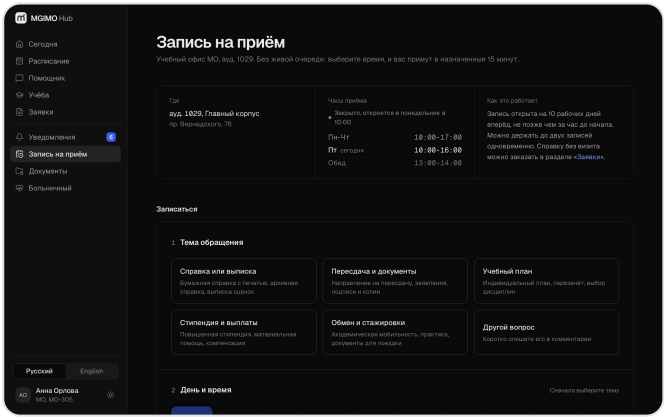
«Документы»: электронный студбилет и все готовые справки с QR, открываются без сети.



Проверка подлинности по QR без входа: её открывает любой, кому предъявили справку. Видны только инициалы.



«Больничный»: сообщить в день болезни, увидеть затронутые пары, получить письмо преподавателю от Атташе.

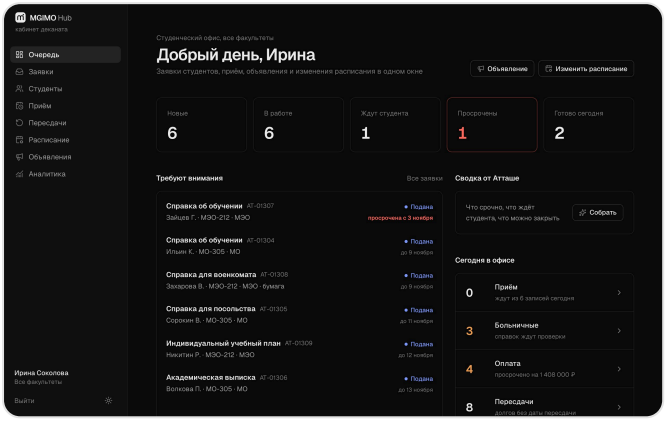


Запись на приём вместо живой очереди: тема, день, свободный слот и талон с QR.

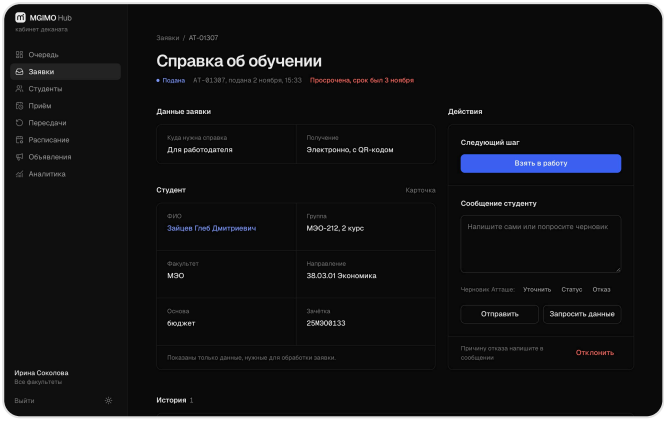
02 · Кабинет деканата

Рабочее место учебного офиса

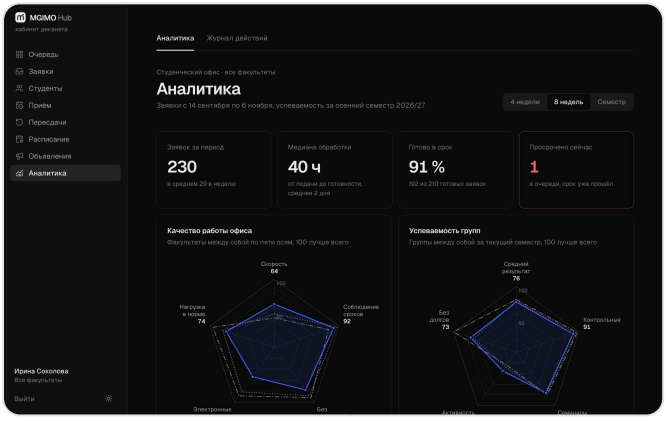
Сотрудник видит только свой факультет. Каждое действие сразу отражается у студента уведомлением и записывается в журнал, который нельзя изменить.



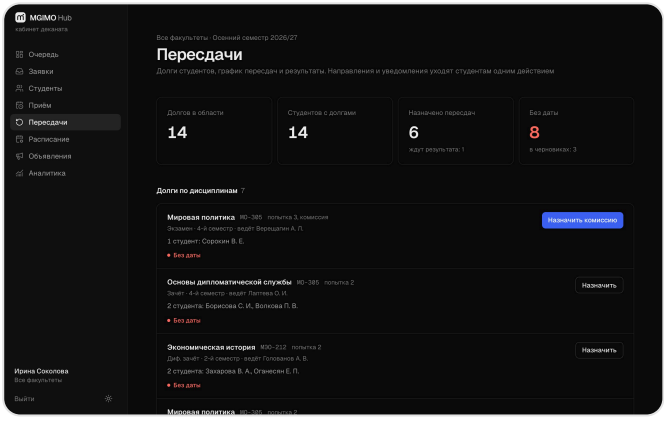
Очередь: что требует внимания, сводка от Атташе и «Сегодня в офисе».



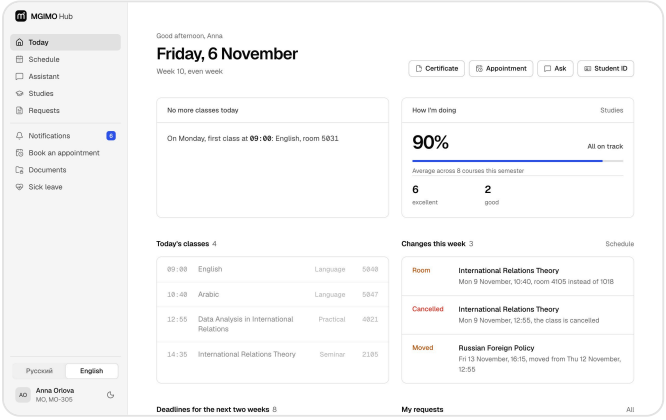
Заявка: действия в один клик, черновик ответа от Атташе, только нужные данные студента.



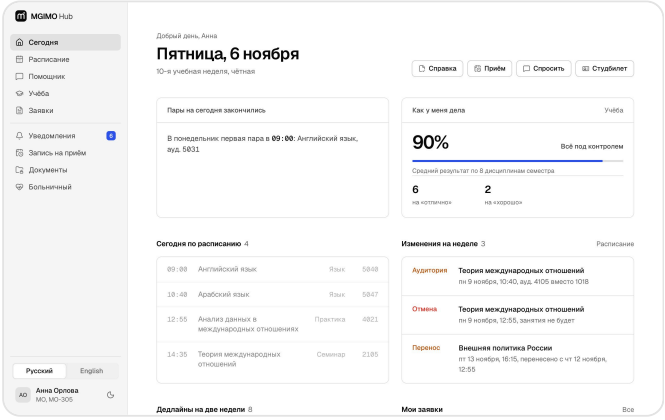
Аналитика: паучьи диаграммы качества работы офиса и успеваемости групп.



Пересдачи одним действием: дата без пересечений с парами, свободная аудитория, направления всем сразу.



Английский интерфейс: уведомления и новости переведены, имена латиницей.



Светлая тема: та же строгая система, усиленный контраст линий и подписей.

Полный перечень ИИ-инструментов и сервисов

Отдельной IDE не было: весь код писал и запускал агент в Claude Code. Фигмы тоже не было: дизайн-система, логотип и анимация чернил на входе сделаны кодом.

Инструмент	Где	Для чего использовали
Claude Code, десктоп-приложение, модель Claude Opus	код, документы	Главный агент: разбор ТЗ и критериев, исследование, архитектура, весь код (29 тыс. строк TypeScript), схема БД и миграции, тексты интерфейса, дизайн-система, тесты, Docker и развёртывание, этот документ
Субагенты Claude Code	код, исследование	12 параллельных агентов: 2 исследователя (факты о МГИМО, боли студентов и бенчмарк) и 10 разработчиков, каждый со своим разделом продукта и своими файлами
Оркестрация Workflow в Claude Code	тестирование	Многоагентный QA: агенты по пяти областям, проверяющий агент после каждого круга, сверка с ТЗ, до трёх кругов исправлений (итоговый проход: 19 агентов, 531 вызов инструментов)
Навыки Claude Code (skills)	дизайн	`redesign`: аудит шаблонных ИИ-паттернов в интерфейсе; `page-load-animations`: рецепты сдержанных анимаций; `workflow-authoring`: сборка оркестрации агентов
Встроенный браузер Claude Code	проверка	Агент сам открывал кабинет, нажимал кнопки, мерил вёрстку на 360 px и снимал скриншоты после каждого крупного блока
Playwright с системным Chrome	проверка	e2e-сценарии (80 прогонов), обход 230 адресов на трёх ширинах, двух темах и двух языках, скриншоты для этого отчёта
DeepSeek V3.2 в Amazon Bedrock	внутри продукта	Модель Атташе: ответы студенту по его данным, черновики заявок и писем, черновики ответов и сводка очереди для деканата, сообщения студентам в зоне риска, перевод объявлений на английский
Память Claude Code	процесс	Постоянные заметки между этапами: решения автора, стиль, обходные пути окружения, чтобы агент не повторял ошибок
Docker, Google Cloud, Caddy, Let's Encrypt, sslip.io	развёртывание	Сборка образа, временный сервер в Финляндии, HTTPS-сертификат и адрес по IP. Всё настроил агент командами gcloud

Тексты интерфейса, демо-данные (61 вымышленный студент, 1 172 занятия, 300 исторических заявок) и английские переводы написаны агентами, автор правил формулировки. Картинки нейросетью не генерировали.

04 · Процесс

Как была выстроена работа с агентами

Главный агент работал как техлид: разбирал задачу, писал документы для других агентов, раздавал работу параллельно и принимал её только после проверок. Человек выступал заказчиком и арт-директором.

1
Постановка
ТЗ, критерии оценки, решения автора по стеку и ИИ

2
Документы для ИИ
CLAUDE.md, PRD, дизайн, безопасность, журнал

3
Параллельные агенты
свой раздел, свои файлы, общая схема БД заранее

4
Проверка
тесты, браузер, проверяющий агент, Docker

5
Ревью автора
замечания по скриншотам, новый круг

Документы, которые написаны для самих агентов

`CLAUDE.md`
Правила проекта, которые каждый агент читает первым: стек, команды, структура, 7 инвариантов безопасности, правила интерфейса автора.

`AGENTS.md`
Предупреждение «это не тот Next.js, что в обучающих данных»: агенты сверяются с документацией версии 16 из `node_modules`.

`docs/PRD.md`
Проблема, принципы, роли, обязательное ядро, смелые идеи, сценарии, которые проверяет жюри.

`docs/DESIGN.md`
Строгий стиль v2: токены цветов, один акцент, шрифты, тон текстов, что запрещено.

`docs/SECURITY.md`
Модель угроз STRIDE, инварианты для разработчиков и агентов, что проверяет автотест.

`docs/PROCESS.md`
Журнал этапов и ошибок ИИ. Агенты дописывали его сами после каждого значимого решения.

Правила, которые позволили агентам работать параллельно

- **Схема базы до запуска агентов.** Все новые таблицы описаны одной миграцией заранее, иначе одновременный `drizzle-kit generate` ломал бы снимки схемы.
- **Владение файлами.** У каждого агента свой список файлов. Проблему в чужом файле агент не чинит, а описывает владельцу в отчёте.
- **Структурированные отчёты.** Агенты возвращают JSON по схеме: что исправлено, что осталось, что передать соседям. Проверяющий агент раздаёт падения владельцам.
- **Свои метки и папки.** Каждый агент помечает тестовые сеансы своим User-Agent и пишет во временную папку, чтобы не выбивать чужие проверки.
- **Готово значит проверено.** Раздел принимается только после типов, линтера, теста безопасности и прогона в браузере.

Ключевые постановки задач по этапам

Слева постановка автора в редакторской правке: формулировки приведены к рабочему виду, требования и порядок сохранены. Справа что сделал агент в ответ.

Старт: ТЗ и рамки

этап 0

Постановка автора

«Участвую в хакатоне «Цифровой деканат будущего», ТЗ прилагаю. Работаю один, своей инфраструктуры нет, развёртывание передам позже. Требования: надёжный бэкенд, проработанный фронтенд, продуманные UX и UI. Ассистент внутри продукта работает на DeepSeek через API, GigaChat не используем.»

Что сделал агент

- Разобрал критерии оценки и обязательное ядро
- Выбрал стек под задачу и упаковку в Docker Compose
- Запустил двух агентов-исследователей параллельно
- Завёл журнал процесса для отчёта

Подключение модели

этап 7

Постановка автора

«Доступ к DeepSeek V3.2 получен через Amazon Bedrock, регион us-east-1. Укажи, где хранить ключ, подключение и проверку выполни самостоятельно.»

Что сделал агент

- Нашёл в документации AWS точный ID модели и Converse API
- Ключ только в переменных окружения сервера, проверка `ai:check` без вывода ключа
- Живой прогон сценариев и попытка получить чужие оценки

Редизайн

этап 8

Постановка автора

«Интерфейс выглядит как типовая генерация нейросети, палитра неудачная. Нужен полный редизайн. Ограничения: без стрелок и длинных тире в текстах, логотип монохромный и выразительнее. Название продукта: MGIMO Hub.»

Что сделал агент

- Разобрал признаки ИИ-шаблона: градиенты, «золото», иконки в плашках, синюшная тёмная тема
- Показал три направления, автор выбрал строгое с кобальтовым акцентом
- Переписал дизайн-систему и все экраны, заменил 134 длинных тире и 20 стрелок

Страница входа

этап 9

Постановка автора

«Страница входа не дотягивает до уровня продукта. Новая композиция: форма справа, анимированное появление логотипа, живые чернила на фоне.»

Что сделал агент

- WebGL-шейдер «тушь в воде», монохромный, с паузой в фоне и статичным кадром для reduced motion
- Сцена: логотип проявляется в центре и переезжает к форме
- Три итерации по скриншотам до плавного варианта

Полировка и ИИ для деканата

этап 10

Постановка автора

«Блоки в профиле не выровнены, интерфейс выглядит сжатым. Добавь сдержанные анимации внутри сервиса с опорой на навыки. Оставь две темы: светлую и тёмную. Деканату нужны генератор справок и доступ к ассистенту.»

Что сделал агент

- Ровная сетка и отступы в 21 файле
- Анимации по навыку page-load-animations, с уважением к reduced motion
- Черновик ответа студенту и сводка очереди от Атташе для сотрудника

05 · Промпты

Продукт, качество, выпуск

Новые разделы

этап 11

Постановка автора

«Исследуй, какие задачи студентов ещё не решены. Обязательно: работа без интернета и push-уведомления, чтобы расписание открывалось в метро; график оплаты и напоминания для платного обучения; английский интерфейс для иностранных студентов; все справки с QR-кодом в одном разделе; сценарий «Я заболел». Отдельно предложи, что ещё полезно деканату.»

Что сделал агент

- Описал всю новую схему БД одной миграцией заранее
- Запустил 10 агентов-разработчиков параллельно, у каждого свой раздел и файлы
- Собрал результат, починил интеграцию, тест безопасности вырос до 46 проверок

Проверка всего продукта

этап 12

Постановка автора

«Проведи полный аудит UX и UI: каждая кнопка и каждый сценарий должны работать. Покрой тестами только ключевые пути, исправь найденное, не больше трёх итераций. Сверь результат с ТЗ и подготовь проект к сдаче.»

Что сделал агент

- Оркестрация: 5 агентов по областям и агент сверки с ТЗ, проверяющий после каждого круга
- Около 40 исправлений, 6 файлов e2e-тестов
- Финал: пересоздание данных, полный прогон, сборка

Ревью по скриншотам

этап 12

Постановка автора

«Блоки разной высоты, на главной приходится прокручивать. Студенческий билет дублируется в профиле и в документах, нужен один источник. Навигация деканата перегружена, «Радар» убрать из меню.»

Что сделал агент

- Нашёл общую причину: колонки разной высоты, перевёл экраны на ряды одинаковой высоты
- Главная стала на 400 px короче, студбилет только в «Документах»
- Меню деканата: 13 пунктов свёрнуты в 8 разделов с вкладками

Английский интерфейс до конца

этап 12

Постановка автора

«Новости деканата и личные уведомления в английской версии остаются на русском. Нужен полный перевод, включая имена.»

Что сделал агент

- Английские поля у объявлений и уведомлений, новое объявление Атташе переводит сам после публикации
- Имена латиницей, как в загранпаспорте, на студбилете и кириллица
- Все тексты уведомлений собраны в одном модуле на двух языках

Выпуск по ссылке

этап 13

Постановка автора

«Разверни временный демо-стенд в Google Cloud с адресом на sslip.io. Отдельный проект, расходы только за счёт кредитов. Для ассистента используй выданный ключ.»

Что сделал агент

- Отдельный проект и бюджет-оповещение на расходы сверх кредитов
- Виртуальная машина, Caddy с сертификатом Let's Encrypt, база закрыта снаружи
- Проверки безопасности против публичного адреса: 46 из 46

Что работало в постановках

Рамки в первой фразе

Цель, ограничения и модель названы сразу: агент не тратит итерацию на догадки о бюджете, составе команды и стеке.

Запреты явно

Стрелки, длинные тире, дисклеймеры, лишние пункты меню перечислены прямо. Вкус автора не приходится угадывать.

Лимит итераций

«Не больше трёх кругов» ограничивает полировку: агент чинит главное и сдаёт результат, а не шлифует бесконечно.

Замечание к экрану

Ревью идёт по скриншотам. Агент ищет общую причину дефекта и чинит её во всех экранах, а не в одном месте.

Сначала исследование

Вопрос «что ещё не решено у студентов» запускает исследование и план, и только потом разработку.

Доступы под контролем

Отдельный проект, расходы только из кредитов, ключи только в окружении сервера и без вывода в журнал.

Как главный агент ставил задачи субагентам

Каждое задание устроено одинаково: роль и контекст, что прочитать до работы, что сделать по шагам, границы файлов, инварианты безопасности и критерии готовности. Ниже фрагменты настоящих заданий, сокращения отмечены многоточием.

Агент-исследователь: боли студентов

этап 1

Роль. Ты UX-исследователь для хакатона «Цифровой деканат будущего». Автор не учится в МГИМО, поэтому опросить однокурсников он не может: нужна кабинетная база из открытых источников.
Границы. Запиши результат в docs/research/student-insights.md. Больше никакие файлы проекта не трогай.
Структура. 1. Ключевые боли (8-12), по каждой 1-2 подтверждения из источников и вывод для дизайна. 2. Бенчмарк 5-7 кабинетов: что позаимствовать, что раздражает. 3. Синтетические персоны с явной пометкой «сгенерировано ИИ». 4. Jobs To Be Done. 5. Выводы для прототипа. 6. Анкета для реальных студентов. 7. Источники.
Правила. Только публичные источники, без персональных данных реальных людей …

Агент-разработчик: больничный

этап 11

Задача. Сейчас студенту после болезни нужна неделя хождений: отнести справку, объяснить пропуски, договориться о переносе контрольных. Делаем один сценарий в кабинете и очередь проверки у деканата.
Сначала прочитай. CLAUDE.md (инварианты), docs/DESIGN.md, docs/SECURITY.md, AGENTS.md: Next.js 16, сверяйся с node_modules/next/dist/docs, про загрузку файлов и лимиты тела запроса тоже посмотри там. Таблицы sick_leaves и sick_leave_files уже есть, миграция применена. **Схему НЕ меняй.**
Паттерны. data/schedule.ts (пары за период), data/staff.ts (область сотрудника: facultyId null значит все факультеты), assistant/providers.ts (вызов модели, офлайн-фолбэк, чистка тире) …
Что сделать. Нет открытого больничного: форма «Сообщить о болезни». Есть открытый: что затрагивает (пары и дедлайны в периоде), для каждой дисциплины «Письмо преподавателю»: черновик от Атташе, офлайн-шаблон, если модели нет …

Оркестрация QA: общее правило для всех агентов

этап 12

ВАЖНО, это хакатон: тестов немного и только на главное. Цель: всё нажимается и работает. В спеке области 4-8 тестов на ключевые сценарии.
Не трогай чужое. Встроенный браузер принадлежит пользователю, скриншоты только через Playwright. Не пересоздавай базу: это сделает финальный этап. Тесты, меняющие данные, должны перезапускаться на той же базе.
Владение файлами. studentCore: вход, оболочка, Сегодня, расписание, учёба, профиль. studentServices: заявки, уведомления, помощник, документы. serviceFlows: приём, больничный, оплата. staffCore и staffInsights: деканат …
Отчёт по схеме. fixed[], remaining[], crossArea[] (проблема в чужом файле с путём и предложением), testsPassed, testsFailed.

Задания субагентам, промпты исследователей и сценарии оркестрации лежат в репозитории и истории проекта: `docs/prompts/`, `docs/PROCESS.md`.

Где ИИ ошибался и как мы это ловили

Каждую ошибку записывали в журнал с тем, как её нашли. Самый частый способ: автотест или живой прогон, а не чтение кода. Полный список из 34 пунктов в `docs/PROCESS.md`.

#	Что пошло не так	Как нашли	Как исправили
1	Знания модели о Next.js устарели: новая версия 16 с другими API	предупреждение в `AGENTS.md`	агент читал документацию версии из `node_modules` до написания кода
8	**Чужая заявка отдавала HTTP 200**: скелетон загрузки начинал ответ раньше проверки доступа	автотест безопасности	у детальных страниц нет скелетона, честный 404
31	Та же ошибка повторилась у разных агентов ещё три раза	e2e-тест на 404 по чужому id	правило в заданиях сработало только вместе с тестом, теперь оба
9	На 360 px главный экран расползался до 525 px	автопроверка ширины	базовый трек сеток `minmax(0,1fr)`
15	Модель отвечала «"отлично" невозможно», хотя впереди ещё 40 баллов	живой прогон вопросов к Атташе	в контекст добавлены «сколько ещё можно набрать» и правило
17	Черновик для студента в зоне риска советовал подать заявление, которое уже подано	живой прогон черновика	в контекст переданы открытые заявки студента
30	Атташе говорил, что дата пересдачи не назначена, хотя направление уже выпущено	вопрос к ассистенту в собранном Docker-образе	в контекст долга добавлены дата, аудитория и номер направления
21	Ось «Без долгов» в аналитике не видела долг: подзапрос ссылался не на ту колонку	сверка диаграммы с SQL	явные алиасы таблиц
24	Защитный заголовок у файла справки перезаписывался глобальным правилом	проверка заголовков curl-ом	отдельное правило для этого пути
34	Модель не всегда давала кнопку черновика справки для визы	e2e-тест ассистента	правило в системном промпте на двух языках

Вывод для процесса. Правило в инструкции агенту снижает число ошибок, но не убирает их. Ошибка перестаёт повторяться, когда на неё есть автотест, поэтому мы превращали каждую найденную проблему безопасности в проверку.

Финальная проверка: три независимых агента

Перед сдачей автор попросил проверить решение целиком. Главный агент запустил трёх проверяющих с чистым контекстом и только с правом чтения, а каждую их находку подтверждал запросом к запущенному серверу, прежде чем править.

Бэкенд и безопасность

Вход, слой данных, серверные действия, ассистент. Нашёл: уведомление вело на внутренний адрес сервера, лимит модели обходился.

Интерфейс

Страницы, переводы, состояния загрузки и ошибки. Нашёл: переписка Атташе оставалась следующему вошедшему, «Повторить» не работала.

Инфраструктура и факты

Docker, тесты, сверка каждой цифры README с кодом. Нашёл: порт базы открыт наружу, в описании кнопка, которой нет в интерфейсе.

Агент-проверяющий: общее правило — этап 13, в переводе

```
Только чтение. Не меняй файлы, не запускай сервер, миграции и сид, не открывай .env.
Нужны настоящие дефекты, а не мнения о стиле. К каждой находке: файл и строка, сценарий «что вводим и что получаем», решающие строки кода.
Отметь уверенность. ПРОВЕРЕНО, если прочитан весь путь кода, или ПРАВДОПОДОБНО. Всё, что не привязано к сценарию, не включай.
В конце. Что сделано хорошо и до чего не дошёл.
```

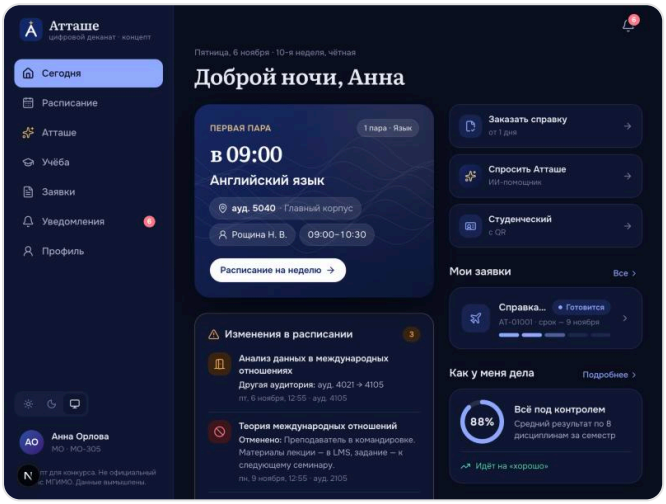
Что нашли	Как подтвердили	Как исправили
Клик по уведомлению вёл на внутренний адрес сервера. Локально этого не видно	запрос с чужим `Host`: в ответе `localhost`	переход относительным адресом
Переписка с Атташе оставалась следующему вошедшему	два агента нашли независимо	история стирается на странице входа
Пять неверных паролей закрывали демо-аккаунт для всех посетителей	чтение кода лимита входа	блокировка по паре «логин и адрес»
Лимит обращений к модели обходился пачкой одновременных запросов	10 запросов разом при 18 занятых из 20: прошли ровно 2	место в лимите занимается до вызова, в транзакции
Один e2e-тест падал: «найдено 2 элемента»	замер по миллисекундам: React 300 мс держит скрытую копию блока	ошибка в тесте, а не в продукте: ожидание в помощнике тестов

Вывод для процесса. Агент плохо видит ошибки в коде, который писал сам: он помнит, что хотел сделать. Проверяющий с чистым контекстом читает то, что написано. А подтверждение запросом к живому серверу отсеивает правдоподобные, но ложные находки: одну такую мы отклонили. После правок: 46 из 46 проверок безопасности и 79 e2e-сценариев.

От «нейрослопа» к строгому стилю

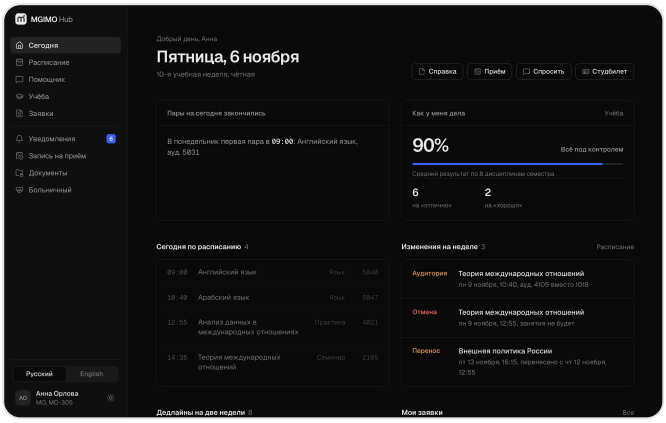
Первую версию автор отклонил: градиенты, «золото», засечки, иконки в цветных плашках, синюшная тёмная тема. Агент разобрал эти признаки шаблонного ИИ-дизайна и пересобрал систему: нейтральные серые, один кобальтовый акцент, шрифт Geist, без теней и декора.

Отклонено



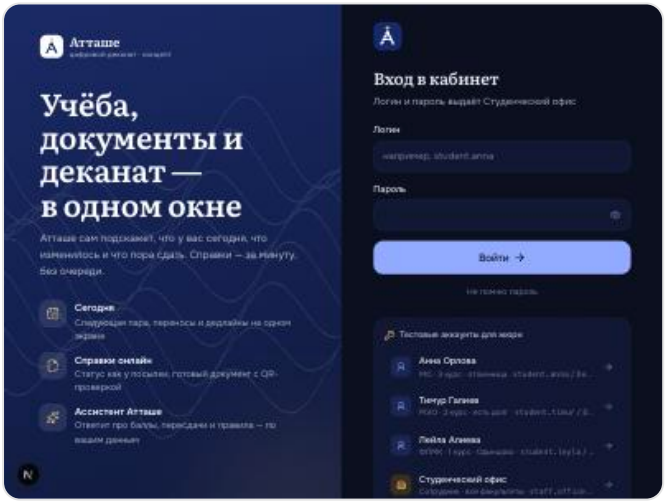
Версия 1: шаблонный ИИ-интерфейс.

Итог



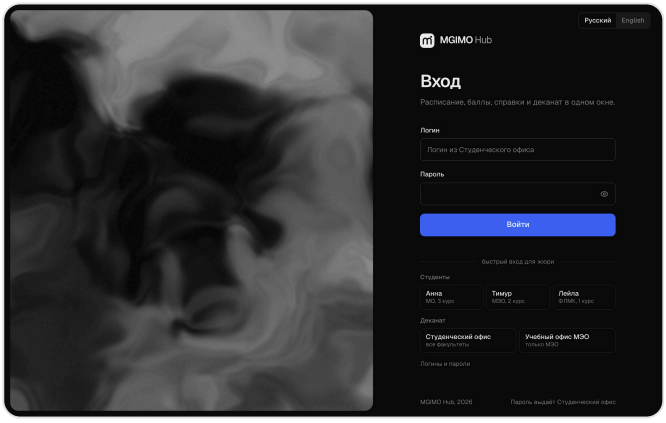
Итоговая версия: строгий стиль, ряды одинаковой высоты.

Отклонено



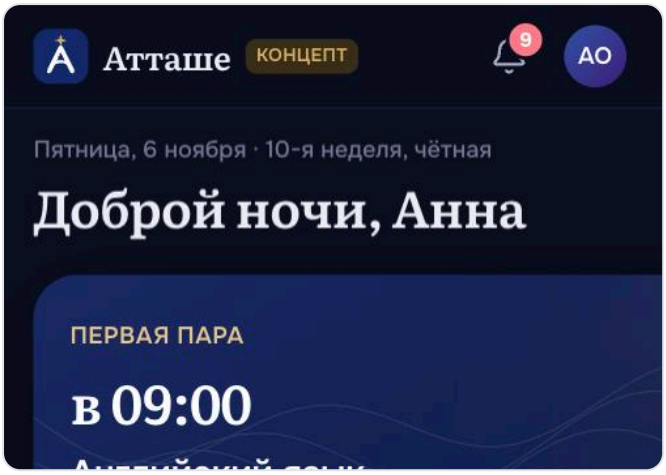
Простая форма входа: «вход не нравится».

Итог



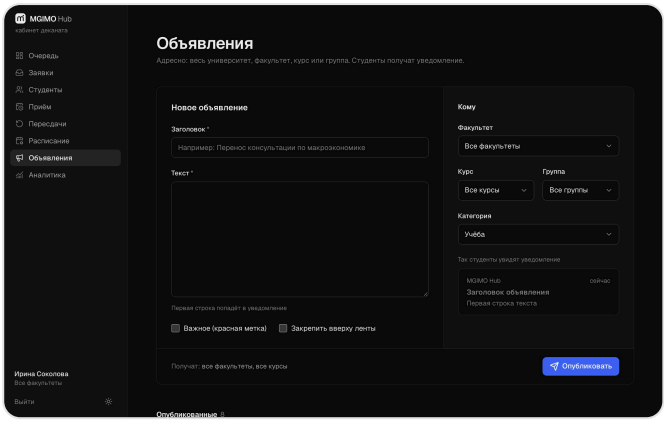
Вход с WebGL-чернилами, три итерации шейдера.

Ошибка



На 360 px экран расползался до 525 px: нашла автопроверка ширины.

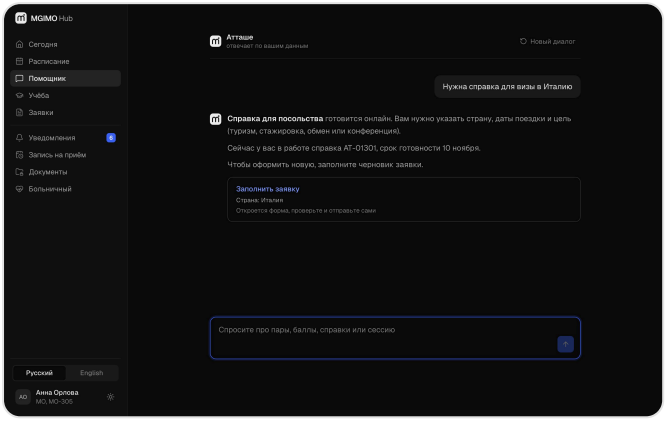
Ревью



Объявления после замечания «пустовато и сжато»: текст слева, кому и предпросмотр уведомления справа.

Атташе: ассистент, который не может навредить

Атташе отвечает по данным самого студента и превращает фразу «нужна справка для визы в Италию» в готовый черновик заявки. Отправляет её всегда студент сам.



Живой ответ модели DeepSeek V3.2 с кнопкой черновика заявки.

Семь сценариев модели

Студент	ответы про пары, баллы, сессию, справки по своим данным
Студент	черновик заявки из обычной фразы
Студент	письмо преподавателю о переносе контрольной при болезни
Деканат	черновик ответа студенту: уточнить, статус, отказ
Деканат	сводка очереди: что срочно, что ждёт студента
Деканат	сообщение студенту в зоне риска
Деканат	перевод объявления на английский после публикации

Как устроена безопасность ассистента

Только свои данные

В контекст модели попадают данные текущего студента и ничего лишнего: без телефона, почты, даты рождения и номера зачётки.

Белый список действий

Модель возвращает ответ и действия. Сервер проверяет каждое: тип заявки, значения полей, свои курсы, только внутренние ссылки.

Работает без модели

Без ключа или при сбое API включается офлайн-режим на правилах, демо не ломается. Ключ хранится только на сервере.

Вход, хранение данных и защита простыми словами

Пароль

Пароль нигде не хранится. В базе лежит только его «отпечаток» argon2id: современный стандарт, медленный для перебора.

Сеанс

После входа браузер получает случайный ключ в защищённой cookie, недоступной скриптам страницы. В базе хранится не сам ключ, а его хэш: даже утечка базы не даст войти.

Только свои данные

Сервер узнаёт студента по сеансу и отдаёт только его записи. Номер заявки в адресе означает лишь «какую из моих». Чужая заявка открывается как «страница не найдена».

Перебор паролей

После 5 неудачных попыток вход по логину блокируется на 15 минут, есть отдельный лимит по IP.

Деканат

Сотрудник видит только свой факультет. Каждый просмотр карточки студента и каждое действие пишутся в журнал, который нельзя изменить или удалить: это запрещено на уровне базы данных.

Ключи сервисов

Ключи ИИ и push-уведомлений лежат только в переменных окружения сервера. В коде и в браузере их нет.

Как это проверено

`npm run test:security` делает 46 проверок против работающего сервера: открыть чужую заявку, справку, дисциплину, талон, квитанцию; зайти студентом в кабинет деканата; сотрудником другого факультета открыть чужую заявку; проверить флаги cookie, заголовки CSP и HSTS, хэши в базе, блокировку после 5 попыток, CSRF у ассистента, минимум данных на странице проверки справки. Последний прогон против публичного адреса: **46 из 46**.

Все студенты, преподаватели, оценки и документы вымышлены. На справках и квитанциях водяной знак «образец», страницы закрыты от поисковиков, логотипа МГИМО нет: у продукта свой знак.

Как устроено внутри

Один Next.js-сервер без состояния и PostgreSQL. Сеансы и лимиты хранятся в базе, поэтому приложение можно запускать в несколько экземпляров за балансировщиком.

Клиент

Браузер и PWA

React 19, установка на экран телефона, свой service worker: расписание, баллы и документы без сети, push.

Студент

11 разделов, русский и английский, светлая и тёмная тема.

Учебный офис

8 разделов, область факультета, журнал действий.

Сервер

Caddy

HTTPS и сертификат Let's Encrypt, наружу только порты 80 и 443.

Next.js 16 · App Router

Серверные компоненты и Server Actions. `proxy.ts` проверяет сеанс. Слой данных `src/lib/data`: каждый запрос начинается с `requireStudent()` или `requireStaff()`, фильтр по id из сеанса.

Route Handlers

ассистент, календарь .ics, файлы справок в песочнице, push-подписки, проверка справки по QR.

Данные и сервисы

PostgreSQL 17 · Drizzle ORM

30 таблиц, 7 миграций. Журнал действий только дописывается: изменение запрещено триггером.

DeepSeek V3.2 · Amazon Bedrock

вызывается только с сервера, офлайн-режим без ключа.

Web Push

VAPID, адреса только известных push-сервисов.

Слой	Что использовано и почему
Фронтенд	Next.js 16, React 19, TypeScript strict, Tailwind CSS 4, свои UI-компоненты, motion для анимаций
Бэкенд	серверные компоненты и Server Actions в том же Next.js: меньше движущихся частей, проверка прав рядом с данными
Данные	PostgreSQL 17 и Drizzle ORM: стабильный, чистый TypeScript, без бинарного движка, проще Docker
Вход	argon2id, случайный токен 256 бит в httpOnly-cookie, в базе SHA-256 токена, лимиты в базе
Валидация	zod на каждом входе: формы, API, ответы модели
Развёртывание	Docker Compose: база, приложение, Caddy. Миграции и демо-данные применяются при старте контейнера

12 · Роль автора

Постановка, решения и приёмка

Код вручную автор не писал. Он отвечал за рамки проекта, продуктовые и визуальные решения, доступы к внешним сервисам и приёмку результата.

Цель и рамки

Сформулировал задачу по ТЗ и ограничения: участие в одиночку, без своей инфраструктуры на старте, DeepSeek как модель ассистента.

Доступы

Получил доступ к DeepSeek V3.2 в Amazon Bedrock и сам внёс ключ в окружение. Определил проект Google Cloud для демо-стенда.

Визуальное направление

Отклонил первую версию интерфейса, из трёх направлений утвердил строгое. Задал правила текстов: без длинных тире, стрелок и дисклеймеров.

Продуктовые решения

Определил состав новых разделов, отказался от рассылок по сегментам, убрал «Радар» из меню, студбилет оставил в «Документах».

Приёмка

Принимал каждую итерацию на ноутбуке и телефоне, фиксировал дефекты скриншотами: неровная сетка, сжатые экраны, дрожание фона.

Сдача

Проверка с российского мобильного интернета, запись демо-видео, отправка работы через личный кабинет хакатона.

13 · Как проверить

Главный путь за 5 минут

#	Шаг	Что увидеть
1	Вход как `student.anna`	«Сегодня»: какая пара идёт, смена аудитории, дедлайны, статус справки для посольства
2	«Расписание»	неделя с переносом, отменой и другой аудиторией, экспорт в календарь
3	«Учёба»	«как у меня дела», рейтинг, калькулятор «что если» в дисциплине
4	«Заявки», «Справка об обучении»	форма с данными из профиля, статус «Подана», срок, трекер
5	«Атташе», «Нужна справка для визы»	ответ по своим данным и кнопка готового черновика
6	Выход, вход как `staff.office`	заявка Анны в очереди, «Взять в работу», «Готово», и у Анны сразу уведомление и справка с QR
7	Подставить чужой id в адрес	«страница не найдена»: чужие данные не открываются
8	Открыть с телефона	всё то же на 360 px, установка на экран, офлайн-расписание